

MCP

Model Context Protocol — Standardised Tool & Data Access for AI Models

Introduction

MCP (Model Context Protocol) is an open protocol, introduced by Anthropic, that standardises how AI applications connect language models to external tools, data sources, and prompt templates. Before MCP, every integration between an LLM app and an external system (a database, an API, a file store) had to be hand-built and was specific to that one app. MCP defines a common client-server protocol — an MCP server exposes Tools (actions), Resources (read-only data), and Prompts (reusable instruction templates); any MCP-compatible client (or LLM agent) can discover and use them without custom integration code.

MCP's defining idea is the separation of capability from consumption: a server built once (e.g., an insurance backend) can be discovered and used by any compliant client, over any transport (stdio for local processes, HTTP/SSE for remote servers), via a consistent handshake of `initialize` -> `list_tools` -> `call_tool`. This makes MCP especially valuable for building agentic AI systems where an LLM needs real, live access to business systems rather than guessing or hallucinating answers.

Components

Component	Description
MCP Server	A process that exposes Tools, Resources, and Prompts over the MCP protocol (here, via FastMCP).
MCP Client	The consumer side of the protocol — connects to a server, discovers its capabilities, and invokes them.
FastMCP	A high-level Python class/decorator API for quickly building an MCP server.
Tool (<code>@mcp.tool()</code>)	An action the server can perform; schema (name, params, description) is inferred from the function signature/docstring.
Resource (<code>@mcp.resource(uri)</code>)	Read-only data addressed by a URI (e.g., <code>policy://all</code>), separate from action-performing tools.
Prompt (<code>@mcp.prompt()</code>)	A reusable, server-defined instruction template that guides how an LLM should approach a task.
ClientSession	The client-side object representing an active MCP session with a server.
<code>stdio_client</code> / <code>StdioServerParameters</code>	Transport that launches a server as a subprocess and talks to it over <code>stdin/stdout</code> .
<code>initialize()</code>	The MCP handshake that establishes a session between client and server.
<code>list_tools()</code> / <code>call_tool()</code>	Real MCP requests: discover available tools, then invoke one by name with arguments.

Syntax

```
# -- SERVER SIDE -----
from mcp.server.fastmcp import FastMCP
mcp = FastMCP("<Server Name>")

@mcp.tool()
def my_tool(param: str) -> str:
    """Docstring becomes the tool's description for the LLM."""
    return f"result for {param}"

@mcp.resource("myscheme://all")
def my_resource() -> str:
    """Read-only data, addressed by URI instead of being 'called'."""
    return "<data>"

@mcp.prompt()
def my_prompt() -> str:
    """A reusable instruction template."""
    return "<step-by-step guidance for the LLM>"

if __name__ == "__main__":
    mcp.run() # starts the server, listens for client connections

# -- CLIENT SIDE -----
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

server_params = StdioServerParameters(command="python", args=["my_server.py"])

async def main():
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize() # handshake
            tools = await session.list_tools() # discovery
            result = await session.call_tool( # invocation
                "my_tool", arguments={"param": "value"})
            print(result.content)

asyncio.run(main())
```

Learning Outcomes

- Explain why AI models need a standardised protocol to access external tools and data.
- Design a well-formed tool schema (name, description, typed parameters) before writing any server code.
- Build a real MCP server using FastMCP, exposing Tools, Resources, and Prompts.
- Build a real MCP client using ClientSession + stdio_client to perform the initialize/list_tools/call_tool sequence.
- Integrate a local LLM (Qwen 2.5 via Ollama) that dynamically selects and calls MCP tools based on a user query.
- Differentiate Tools, Resources, and Prompts and choose the right primitive for a given capability.

Software Requirements

Requirement	Detail
Python Version	3.10 or higher
Core Package	mcp (official MCP Python SDK, includes server.fastmcp and client.stdio)
Other Libraries	httpx (HTTP calls to Ollama), python-dotenv, rich (console output)
LLM Backend	Ollama running locally, serving qwen2.5:3b (capstone only)
API Key	None required — fully local via Ollama
Environment	Any local Python environment, terminal, or Jupyter/VS Code setup
Install Command	<code>pip install -r requirements3.txt</code> (mcp, httpx, python-dotenv, rich)

Lab Programs

Lab 1: Understanding MCP Basics (Plain Python Function)

Source Code

```
# =====
# HANDS-ON 1 -- Understanding MCP Basics
# =====

def get_policy_details(policy_id: str) -> str:
    """Look up policy type by policy ID."""
    policies = {
        "P1001": "Health Insurance",
        "P1002": "Car Insurance",
        "P1003": "Life Insurance",
    }
    return policies.get(policy_id, "Policy Not Found")

print("Calling get_policy_details('P1002') directly:")
print(" ", get_policy_details("P1002"))

print("\nCalling get_policy_details('P9999') (unknown ID):")
print(" ", get_policy_details("P9999"))
```

Sample Input / Output

```
get_policy_details("P1002")
get_policy_details("P9999")

--- Output ---
Calling get_policy_details('P1002') directly:
```

Car Insurance

```
Calling get_policy_details('P9999') (unknown ID):  
Policy Not Found
```

Key Concepts Covered

Concept	Description
Plain Function Call	A regular Python function with no protocol involved — the baseline MCP improves upon.
The Core Problem	An LLM cannot see or call this function directly; it has no access to your source code.
Motivation for MCP	Sets up why a standardised way to expose tools to an AI model is needed.

Lab 2: MCP Tool Registry & Discovery (Simulated)

Source Code

```
# =====
# HANDS-ON 2 -- MCP TOOL REGISTRY & DISCOVERY
# =====

print("\n=== MCP TOOL DISCOVERY ===\n")

# -- STEP 1: Define MCP Tools -----

def get_policy(policy_id):
    return f"Policy Details for {policy_id}"

def claim_status(claim_id):
    return f"Claim Status for {claim_id}"

def calculate_premium(age):
    if age < 30:
        return 5000
    elif age < 50:
        return 8000
    else:
        return 12000

# -- STEP 2: Simulate MCP Server Tool Registry -----

tool_registry = {
    "get_policy": get_policy,
    "claim_status": claim_status,
    "calculate_premium": calculate_premium
}

# -- STEP 3: MCP Client Discovers Available Tools -----

print("Tools discovered from MCP Server:\n")
for tool_name in tool_registry:
    print("-", tool_name)

# -- STEP 4: MCP Client Calls a Tool -----

selected_tool = "claim_status"
print("\nClient selected tool:", selected_tool)
result = tool_registry[selected_tool]("C1001")
print("Tool Result:", result)

# -- STEP 5: Call Another Tool -----

selected_tool = "calculate_premium"
print("\nClient selected tool:", selected_tool)
result = tool_registry[selected_tool](35)
print("Tool Result:", result)
```

Sample Input / Output

```
tool_registry["claim_status"]("C1001")
tool_registry["calculate_premium"](35)
```

--- Output ---

Tools discovered from MCP Server:

- get_policy
- claim_status

```
- calculate_premium
```

```
Client selected tool: claim_status  
Tool Result: Claim Status for C1001
```

```
Client selected tool: calculate_premium  
Tool Result: 8000
```

Key Concepts Covered

Concept	Description
Tool Registry	A dictionary mapping tool names to functions — a simplified stand-in for an MCP server.
Tool Discovery	The client iterates the registry to see what tools are available, before calling any of them.
Name-Based Invocation	Tools are called dynamically by string name (registry[name](args)), not hard-coded calls.

Lab 3: Designing a Tool Schema

Source Code

```
# =====
# HANDS-ON 3 -- Design a Tool Schema
# =====

import json

print("\n=== HANDS-ON 3: DESIGNING AN MCP TOOL SCHEMA ===\n")

# -- STEP 1: Why Schemas Matter -----
print("=" * 60)
print("WHY DOES A TOOL NEED A SCHEMA?")
print("=" * 60)
print("""
An AI model cannot read your Python source code to know:
- What does this tool do?
- What parameters does it need?
- What TYPE should each parameter be?

The schema is a structured description the AI reads BEFORE
calling the tool. It's like a function's "API documentation"
but in a format machines can parse and reason over.
""")

# -- STEP 2: Build a Tool Schema -- claim_status -----

claim_status_tool = {
    "name": "claim_status",
    "description": "Get insurance claim status",
    "parameters": {
        "claim_id": {"type": "string", "description": "Claim Identifier"}
    }
}

print("=" * 60)
print("SCHEMA: claim_status_tool")
print("=" * 60)
print(json.dumps(claim_status_tool, indent=2))

# -- STEP 3: Build 2 More Schemas -- Practice -----

policy_lookup_tool = {
    "name": "get_policy",
    "description": "Retrieve policy details by policy ID",
    "parameters": {
        "policy_id": {"type": "string", "description": "Unique policy identifier, e.g. P1001"}
    }
}

premium_calculator_tool = {
    "name": "calculate_premium",
    "description": "Calculate insurance premium based on customer age and policy type",
    "parameters": {
        "age": {"type": "integer", "description": "Customer age in years"},
        "policy_type": {"type": "string", "description": "Type of policy: Health, Car, or Life"}
    }
}

print("\n" + "=" * 60)
print("SCHEMA: get_policy")
print("=" * 60)
print(json.dumps(policy_lookup_tool, indent=2))

print("\n" + "=" * 60)
```

```

print("SCHEMA: calculate_premium (multiple parameters)")
print("=" * 60)
print(json.dumps(premium_calculator_tool, indent=2))

# -- STEP 4: Validate a Schema (basic structure check) -----

def validate_tool_schema(schema: dict) -> bool:
    """Check that a tool schema has the minimum required structure."""
    required_top_level = ["name", "description", "parameters"]
    for field in required_top_level:
        if field not in schema:
            print(f"  INVALID: missing top-level field '{field}'")
            return False
    for param_name, param_def in schema["parameters"].items():
        if "type" not in param_def:
            print(f"  INVALID: parameter '{param_name}' missing 'type'")
            return False
    print(f"  VALID: '{schema['name']}' schema is well-formed")
    return True

print("\n" + "=" * 60)
print("VALIDATING ALL 3 SCHEMAS")
print("=" * 60)
validate_tool_schema(claim_status_tool)
validate_tool_schema(policy_lookup_tool)
validate_tool_schema(premium_calculator_tool)

broken_schema = {"name": "broken_tool", "parameters": {}}
print("\nTesting an intentionally broken schema (missing 'description'):")
validate_tool_schema(broken_schema)

```

Sample Input / Output

```

validate_tool_schema(claim_status_tool)
validate_tool_schema({"name": "broken_tool", "parameters": {}})

--- Output ---
SCHEMA: claim_status_tool
{
  "name": "claim_status",
  "description": "Get insurance claim status",
  "parameters": {"claim_id": {"type": "string", "description": "Claim Identifier"}}
}

VALID: 'claim_status' schema is well-formed

Testing an intentionally broken schema (missing 'description'):
INVALID: missing top-level field 'description'

```

Key Concepts Covered

Concept	Description
Tool Schema	A structured JSON description (name, description, parameters) an AI reads before calling a tool.
Why Schemas Matter	An LLM cannot read your source code — the schema is the tool's machine-readable documentation.
Schema Validation	Checking that name/description/parameters exist and every parameter declares a type.

Lab 4: Your First Real MCP Server

Source Code

```
# =====
# HANDS-ON 4 -- Your First Real MCP Server
# =====

from mcp.server.fastmcp import FastMCP

# -- STEP 1: Create the MCP Server -----

mcp = FastMCP("Insurance Server")

# -- STEP 2: Define a Tool with @mcp.tool() -----

@mcp.tool()
def get_policy(policy_id: str) -> str:
    """Get insurance policy type by policy ID."""
    policies = {
        "P1001": "Health Insurance",
        "P1002": "Car Insurance",
        "P1003": "Life Insurance",
    }
    return policies.get(policy_id, "Not Found")

# -- STEP 3: Start the Server -----

if __name__ == "__main__":
    print("Starting Insurance MCP Server...")
    print("Tool available: get_policy(policy_id)")
    print("Waiting for MCP client connections (Ctrl+C to stop)...\n")
    mcp.run()
```

Sample Input / Output

```
# Run: python 4_First_Server.py
# (a separate MCP client would call get_policy("P1002") over the protocol)

--- Output ---
Starting Insurance MCP Server...
Tool available: get_policy(policy_id)
Waiting for MCP client connections (Ctrl+C to stop)...
```

Key Concepts Covered

Concept	Description
FastMCP	A high-level Python class that turns plain functions into a real, running MCP server.
@mcp.tool()	Decorator that registers a function as a callable MCP tool, auto-building its schema from the signature and docstring.
mcp.run()	Starts the server and listens for MCP client connections over a transport (stdio by default).

Lab 5: Multiple MCP Tools on One Server

Source Code

```
# =====
# HANDS-ON 6 -- Multiple MCP Tools on One Server
# =====

from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Insurance Server")

# -- TOOL 1: Policy Lookup -----

@mcp.tool()
def get_policy(policy_id: str) -> str:
    """Get insurance policy details by policy ID."""
    policies = {
        "P1001": "Health Insurance",
        "P1002": "Car Insurance",
        "P1003": "Life Insurance",
    }
    return f"Policy {policy_id}: {policies.get(policy_id, 'Not Found')}"

# -- TOOL 2: Claim Status -----

@mcp.tool()
def claim_status(claim_id: str) -> str:
    """Check the current status of an insurance claim."""
    claims = {
        "C1001": "Approved",
        "C1002": "Under Review",
        "C1003": "Rejected",
    }
    status = claims.get(claim_id, "Claim Not Found")
    return f"Claim {claim_id}: {status}"

# -- TOOL 3: Premium Due -----

@mcp.tool()
def premium_due(customer_id: str) -> str:
    """Get the premium amount due for a customer."""
    premiums = {"CUST001": 5000, "CUST002": 3200, "CUST003": 7800}
    amount = premiums.get(customer_id)
    if amount is None:
        return f"No premium record found for {customer_id}"
    return f"Premium due for {customer_id}: Rs {amount:,"}

# -- Start the Server -----

if __name__ == "__main__":
    print("Starting Insurance MCP Server...")
    print("\n3 Tools available:")
    print("  1. get_policy(policy_id)")
    print("  2. claim_status(claim_id)")
    print("  3. premium_due(customer_id)")
    print("\nWaiting for MCP client connections (Ctrl+C to stop)...\n")
    mcp.run()
```

Sample Input / Output

```
# Server exposes 3 tools: get_policy, claim_status, premium_due

--- Output ---
```

```
Starting Insurance MCP Server...
```

```
3 Tools available:
```

1. `get_policy(policy_id)`
2. `claim_status(claim_id)`
3. `premium_due(customer_id)`

```
Waiting for MCP client connections (Ctrl+C to stop)...
```

Key Concepts Covered

Concept	Description
Multiple Tool Registration	Several <code>@mcp.tool()</code> -decorated functions can coexist on a single FastMCP server.
Independent Tool Schemas	Each tool's parameters and description are inferred separately from its own signature/docstring.
Single Server, Many Capabilities	One server process can expose an entire domain's worth of actions.

Lab 6: MCP Resources

Source Code

```
# =====
# HANDS-ON 7 -- MCP Resources
# =====

from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Insurance Server")

# -- TOOLS (actions) -- same as Hands-on 6 -----

@mcp.tool()
def get_policy(policy_id: str) -> str:
    """Get insurance policy details by policy ID."""
    policies = {
        "P1001": "Health Insurance",
        "P1002": "Car Insurance",
        "P1003": "Life Insurance",
    }
    return policies.get(policy_id, "Not Found")

@mcp.tool()
def claim_status(claim_id: str) -> str:
    """Check the current status of an insurance claim."""
    claims = {"C1001": "Approved", "C1002": "Under Review", "C1003": "Rejected"}
    return claims.get(claim_id, "Claim Not Found")

# -- RESOURCE 1: All Policies (read-only data) -----

@mcp.resource("policy://all")
def get_all_policies() -> str:
    """Provide the full list of available insurance policies."""
    return """
P1001 - Health Insurance
P1002 - Car Insurance
P1003 - Life Insurance
"""

# -- RESOURCE 2: Company FAQ (another read-only data example) -----

@mcp.resource("insurance://faq")
def get_faq() -> str:
    """Provide frequently asked questions about insurance policies."""
    return """
Q: How long does claim approval take?
A: 5-7 business days for standard claims.

Q: Can I change my policy after purchase?
A: Yes, within the first 15 days (free-look period).

Q: What documents are needed for a claim?
A: ID proof, policy document, and incident report.
"""

# -- RESOURCE 3: Customer-Specific Data (parameterised resource) -----

@mcp.resource("customer://{customer_id}/summary")
def get_customer_summary(customer_id: str) -> str:
    """Provide a summary of a specific customer's account."""
    customers = {
        "CUST001": "John Doe - Health Insurance - Active - Premium Rs 5000/yr",
        "CUST002": "Jane Smith - Car Insurance - Active - Premium Rs 3200/yr",
    }
    return customers.get(customer_id, "Customer not found")
```

```

    return customers.get(customer_id, f"No record found for {customer_id}")

# -- Start the Server -----

if __name__ == "__main__":
    print("Starting Insurance MCP Server...")
    print("\n2 Tools (actions):")
    print("  1. get_policy(policy_id)")
    print("  2. claim_status(claim_id)")
    print("\n3 Resources (data):")
    print("  1. policy://all")
    print("  2. insurance://faq")
    print("  3. customer://{customer_id}/summary")
    print("\nWaiting for MCP client connections (Ctrl+C to stop)...\n")
    mcp.run()

```

Sample Input / Output

```

# Resources exposed: policy://all, insurance://faq, customer://{customer_id}/summary

--- Output ---
Starting Insurance MCP Server...

2 Tools (actions):
  1. get_policy(policy_id)
  2. claim_status(claim_id)

3 Resources (data):
  1. policy://all
  2. insurance://faq
  3. customer://{customer_id}/summary

```

Key Concepts Covered

Concept	Description
MCP Resource	Read-only data exposed via a URI (e.g., policy://all), distinct from an action-performing Tool.
@mcp.resource(uri)	Decorator that registers a function as a resource, addressable by its URI pattern.
Parameterised Resource	customer://{customer_id}/summary shows a URI template with a dynamic path segment.

Lab 7: MCP Prompts

Source Code

```
# =====
# HANDS-ON 8 -- MCP Prompts
# =====

from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Insurance Server")

# -- TOOLS (from previous hands-on labs) -----

@mcp.tool()
def get_policy(policy_id: str) -> str:
    """Get insurance policy details by policy ID."""
    policies = {"P1001": "Health Insurance", "P1002": "Car Insurance", "P1003": "Life Insurance"}
    return policies.get(policy_id, "Not Found")

@mcp.tool()
def claim_status(claim_id: str) -> str:
    """Check the current status of an insurance claim."""
    claims = {"C1001": "Approved", "C1002": "Under Review", "C1003": "Rejected"}
    return claims.get(claim_id, "Claim Not Found")

# -- RESOURCE (from Hands-on 7) -----

@mcp.resource("policy://all")
def get_all_policies() -> str:
    """Provide the full list of available insurance policies."""
    return "P1001 - Health Insurance\nP1002 - Car Insurance\nP1003 - Life Insurance"

# -- PROMPT 1: Claim Review Workflow -----

@mcp.prompt()
def claim_review_prompt() -> str:
    """A structured workflow for reviewing an insurance claim."""
    return """
Review the claim details carefully and follow these steps:

1. Verify the claim_id exists and pull its current status
2. Check the policy is active and covers the claimed incident
3. Identify any fraud indicators:
   - Unusually high claim amount vs policy history
   - Claim filed very soon after policy purchase
   - Multiple claims from the same customer in a short period
4. Summarise your findings in plain language
5. Recommend: Approve / Investigate Further / Reject
"""

# -- PROMPT 2: Customer Communication Template -----

@mcp.prompt()
def customer_response_prompt() -> str:
    """A tone and structure guide for responding to customer queries."""
    return """
When responding to insurance customers:

1. Acknowledge their question warmly and clearly
2. Provide the specific answer using available tools/resources
   (do not guess -- always look up real claim/policy data)
3. Explain any next steps in plain, non-technical language
4. Avoid insurance jargon unless the customer used it first
5. End with an offer to help further if needed
"""
```

```
# -- PROMPT 3: Policy Renewal Reminder -----

@mcp.prompt()
def renewal_reminder_prompt() -> str:
    """A template for generating policy renewal reminders."""
    return """
    When generating a renewal reminder for a customer:

    1. State the policy type and policy ID clearly
    2. State the exact renewal/expiry date
    3. State the premium amount due
    4. Mention any late renewal penalties if applicable
    5. Provide a clear call to action (how to renew)
    Keep the tone friendly but create appropriate urgency if
    the renewal date is within 7 days.
    """

# -- Start the Server -----

if __name__ == "__main__":
    print("Starting Insurance MCP Server...")
    print("\n2 Tools:")
    print("  1. get_policy(policy_id)")
    print("  2. claim_status(claim_id)")
    print("\n1 Resource:")
    print("  1. policy://all")
    print("\n3 Prompts:")
    print("  1. claim_review_prompt")
    print("  2. customer_response_prompt")
    print("  3. renewal_reminder_prompt")
    print("\nWaiting for MCP client connections (Ctrl+C to stop)...\n")
    mcp.run()
```

Sample Input / Output

```
# Prompts exposed: claim_review_prompt, customer_response_prompt, renewal_reminder_prompt

--- Output ---
Starting Insurance MCP Server...

2 Tools: get_policy, claim_status
1 Resource: policy://all
3 Prompts:
  1. claim_review_prompt
  2. customer_response_prompt
  3. renewal_reminder_prompt
```

Key Concepts Covered

Concept	Description
MCP Prompt	A reusable, server-defined instruction template that guides how an LLM should approach a task.
@mcp.prompt()	Decorator that registers a function whose return value is a structured prompt template.
Tools vs Resources vs Prompts	Tools = actions, Resources = data, Prompts = reusable instruction templates — MCP's three primitives.

Lab 8: Capstone — Insurance MCP Server + Client with Qwen Agent

Source Code — insurance_server.py

```
# =====
# MCP CAPSTONE -- Part 1: Insurance MCP Server
# =====

from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Insurance MCP Server")

# -- DATA LAYER -- the actual business data behind each tool -----

CUSTOMER_DB = {
    "C001": {"name": "John", "policy_id": "P1001", "since": "2021", "email": "john@example.com"},
    "C002": {"name": "Priya", "policy_id": "P1002", "since": "2023", "email": "priya@example.com"},
    "C003": {"name": "Ravi", "policy_id": "P1003", "since": "2020", "email": "ravi@example.com"},
}

POLICY_DB = {
    "P1001": {"type": "Health Insurance", "premium": 5000, "active": True, "coverage": 200000},
    "P1002": {"type": "Car Insurance", "premium": 3200, "active": True, "coverage": 100000},
    "P1003": {"type": "Life Insurance", "premium": 7800, "active": False, "coverage": 500000},
}

CLAIM_DB = {
    "CL001": {"customer_id": "C001", "policy_id": "P1001", "status": "Approved", "amount": 25000},
    "CL002": {"customer_id": "C002", "policy_id": "P1002", "status": "Under Review", "amount": 80000},
    "CL003": {"customer_id": "C001", "policy_id": "P1001", "status": "Rejected", "amount": 150000},
    "CL004": {"customer_id": "C003", "policy_id": "P1003", "status": "Submitted", "amount": 35000},
}

# -- THE 6 MCP TOOLS -----

@mcp.tool()
def get_customer(customer_id: str) -> str:
    """Get full customer profile by customer ID. Args: customer_id e.g. C001, C002, C003"""
    record = CUSTOMER_DB.get(customer_id)
    if not record:
        return f"ERROR: Customer {customer_id} not found."
    return (f"Customer {customer_id}: Name={record['name']}, Policy={record['policy_id']}, "
            f"Member since {record['since']}, Email={record['email']}")

@mcp.tool()
def get_policy(policy_id: str) -> str:
    """Get insurance policy details by policy ID. Args: policy_id e.g. P1001, P1002, P1003"""
    record = POLICY_DB.get(policy_id)
    if not record:
        return f"ERROR: Policy {policy_id} not found."
    status = "Active" if record["active"] else "Inactive"
    return (f"Policy {policy_id}: Type={record['type']}, Status={status}, "
            f"Annual Premium=Rs {record['premium']:,}, Coverage=Rs {record['coverage']:,}")

@mcp.tool()
def get_claim_status(claim_id: str) -> str:
    """Get the current status of an insurance claim. Args: claim_id e.g. CL001..CL004"""
    record = CLAIM_DB.get(claim_id)
    if not record:
        return f"ERROR: Claim {claim_id} not found."
    return (f"Claim {claim_id}: Status={record['status']}, Amount=Rs {record['amount']:,}, "
            f"Customer={record['customer_id']}, Policy={record['policy_id']}")

@mcp.tool()
def get_premium_due(customer_id: str) -> str:
    """Get the annual premium amount due for a customer. Args: customer_id e.g. C001..C003"""
```

```

customer = CUSTOMER_DB.get(customer_id)
if not customer:
    return f"ERROR: Customer {customer_id} not found."
policy = POLICY_DB.get(customer["policy_id"])
if not policy:
    return f"ERROR: No policy found for {customer_id}."
return (f"Premium due for {customer_id} ({customer['name']}): "
        f"Rs {policy['premium']:,} per year for {policy['type']}")

@mcp.tool()
def run_fraud_check(claim_id: str) -> str:
    """Run a fraud risk assessment on an insurance claim. Args: claim_id e.g. CL001..CL004"""
    claim = CLAIM_DB.get(claim_id)
    if not claim:
        return f"ERROR: Claim {claim_id} not found."
    amount = claim["amount"]
    if amount > 100000:
        risk, action, score = "HIGH RISK", "Manual review required before processing", 85
    elif amount > 50000:
        risk, action, score = "MEDIUM RISK", "Standard verification recommended", 55
    else:
        risk, action, score = "LOW RISK", "Proceed with normal processing", 15
    return (f"Fraud check for {claim_id}: Risk={risk}, Score={score}/100, "
            f"Amount=Rs {amount:,}, Action={action}")

@mcp.tool()
def renew_policy(policy_id: str) -> str:
    """Renew an inactive insurance policy. Args: policy_id to renew, e.g. P1001..P1003"""
    policy = POLICY_DB.get(policy_id)
    if not policy:
        return f"ERROR: Policy {policy_id} not found."
    if policy["active"]:
        return f"Policy {policy_id} is already active -- no renewal needed."
    policy["active"] = True
    return (f"Policy {policy_id} renewed successfully. Type={policy['type']}, "
            f"Premium=Rs {policy['premium']:,}, Coverage=Rs {policy['coverage']:,}")

# -- RUN THE SERVER -----

if __name__ == "__main__":
    print("Starting Insurance MCP Server...")
    print("6 tools available:")
    print(" 1. get_customer(customer_id)")
    print(" 2. get_policy(policy_id)")
    print(" 3. get_claim_status(claim_id)")
    print(" 4. get_premium_due(customer_id)")
    print(" 5. run_fraud_check(claim_id)")
    print(" 6. renew_policy(policy_id)")
    print("\nWaiting for MCP client connections (Ctrl+C to stop)...\n")
    try:
        mcp.run()
    except KeyboardInterrupt:
        print("\nServer stopped.")

```

Source Code — insurance_client.py

```

# =====
# MCP CAPSTONE -- Part 2: Insurance MCP Client + Qwen Agent
# =====
# Real MCP client: uses ClientSession + stdio_client to talk to
# insurance_server.py over the actual MCP protocol:
#   initialize -> handshake with the server
#   list_tools -> discover all 6 tools automatically
#   call_tool -> invoke a specific tool by name
#
# ARCHITECTURE:
#   User query -> Qwen (Ollama) decides which tool(s) to call

```

```

# -> MCP Client (ClientSession + stdio_client) [REAL MCP protocol]
# -> insurance_server.py (FastMCP server) -> tool executes
# -> result returns through the protocol -> Qwen synthesizes answer
# =====

import asyncio
import json
import os
import httpx
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from rich.console import Console
from rich.panel import Panel
from rich.table import Table

console = Console()

OLLAMA_URL = "http://localhost:11434/api/chat"
MODEL_NAME = "qwen2.5:3b"
SERVER_PATH = os.path.join(os.path.dirname(__file__), "insurance_server.py")

# -- SECTION 1: QWEN VIA OLLAMA -----

def call_qwen(prompt: str) -> str:
    """Send a prompt to Qwen via Ollama's REST API."""
    payload = {"model": MODEL_NAME, "messages": [{"role": "user", "content": prompt}], "stream":
False}
    try:
        response = httpx.post(OLLAMA_URL, json=payload, timeout=60)
        response.raise_for_status()
        return response.json()["message"]["content"]
    except httpx.ConnectError:
        return "ERROR: Cannot connect to Ollama. Is it running? Try: ollama serve"
    except Exception as e:
        return f"ERROR: {e}"

# -- SECTION 2: REAL MCP TOOL DISCOVERY -----

async def discover_tools(session: ClientSession) -> dict:
    """Call session.list_tools() -- the real MCP tools/list request."""
    tools_result = await session.list_tools()
    tool_map = {}
    for tool in tools_result.tools:
        tool_map[tool.name] = tool.description or "No description"
    return tool_map

# -- SECTION 3: QWEN DECIDES WHICH TOOLS TO CALL -----

def decide_tools(query: str, available_tools: dict) -> list:
    tools_description = "\n".join(
        f"{i+1}. {name}: {desc}" for i, (name, desc) in enumerate(available_tools.items())
    )
    prompt = f"""
You are an insurance assistant.

Available tools:
{tools_description}

Available customer IDs: C001, C002, C003
Available policy IDs: P1001, P1002, P1003
Available claim IDs: CL001, CL002, CL003, CL004

User Query:
{query}

IMPORTANT:
- Return ONLY JSON, no markdown, no explanation

```

- Always return a JSON array

Example:

```
[{"tool": "get_claim_status", "input": {"claim_id": "CL001"}}]
```

```
"""
```

```
raw = call_qwen(prompt)
console.print("\n[cyan]RAW QWEN RESPONSE:[/cyan]")
console.print(raw)
```

```
cleaned = raw.strip().replace("```json", "").replace("```", "").strip()
```

```
try:
```

```
    result = json.loads(cleaned)
    if isinstance(result, dict):
        result = [result]
    if not isinstance(result, list):
        return []
    return [item for item in result if isinstance(item, dict)]
```

```
except Exception as e:
```

```
    console.print(f"[red]JSON Parse Error:[/red] {e}")
    console.print(f"[yellow]Response:[/yellow] {cleaned}")
    return []
```

```
# -- SECTION 4: REAL MCP TOOL CALLS via session.call_tool() -----
```

```
async def execute_tools_via_mcp(session: ClientSession, tool_calls: list) -> list:
```

```
    results = []
```

```
    for call in tool_calls:
```

```
        if not isinstance(call, dict):
            console.print(f"[red]Skipping invalid tool call:[/red] {call}")
            continue
```

```
        tool_name = call.get("tool", "")
        tool_input = call.get("input", {})
```

```
        if not tool_name:
            console.print(f"[red]Missing tool name[/red]")
            continue
```

```
        console.print(f"[blue]MCP call_tool({tool_name}, {tool_input})[/blue]")
```

```
        try:
```

```
            result = await session.call_tool(tool_name, arguments=tool_input)
            texts = []
            if hasattr(result, "content"):
                for item in result.content:
                    if hasattr(item, "text"):
                        texts.append(item.text)
            output_text = ".join(texts) if texts else "(empty result)"
            console.print(f"[green]Result:[/green] {output_text}")
            results.append({"tool": tool_name, "input": tool_input, "output": output_text})
```

```
        except Exception as e:
```

```
            console.print(f"[red]Tool Error:[/red] {e}")
            results.append({"tool": tool_name, "input": tool_input, "error": str(e)})
```

```
    return results
```

```
# -- SECTION 5: SYNTHESIZE FINAL ANSWER -----
```

```
def synthesize_answer(query: str, tool_results: list) -> str:
```

```
    """Ask Qwen to form a natural language answer from the MCP tool results."""
```

```
    results_text = json.dumps(tool_results, indent=2)
```

```
    prompt = f"""Original question: {query}
```

Real data retrieved via MCP tools:

```
{results_text}
```

Write a clear, professional answer for the customer using this real data.

Include specific numbers and statuses. Keep to 2-4 sentences.

Do not mention MCP, tools, or JSON -- just answer naturally.

```
"""
```

```
    return call_qwen(prompt)
```

```
# -- SECTION 6: MAIN AGENT PIPELINE -----

async def run_insurance_agent(queries: list):
    """
    1. Launch insurance_server.py as a subprocess
    2. Connect via stdio_client + ClientSession
    3. initialize() -> real MCP handshake
    4. list_tools() -> real tool discovery
    5. For each query: Qwen decides tools -> call_tool() -> synthesize
    """
    server_params = StdioServerParameters(command="python", args=[SERVER_PATH])
    console.print("[cyan]Launching MCP server and connecting...[/cyan]")

    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            console.print("[green]MCP handshake complete (initialize)[/green]")

            available_tools = await discover_tools(session)
            console.print(f"[green]Tools discovered (list_tools):\n{list(available_tools.keys())}[/green]\n")

            table = Table(title="Discovered Tools from MCP Server", style="bold")
            table.add_column("Tool", style="cyan", width=22)
            table.add_column("Description", style="white", width=55)
            for name, desc in available_tools.items():
                table.add_row(name, desc[:55] if desc else "-")
            console.print(table)
            console.print()

            for query in queries:
                console.print(f"\n[bold white]Customer Query:[/bold white] {query}")
                console.print("[cyan]Qwen selecting tools...[/cyan]")
                tool_calls = decide_tools(query, available_tools)

                if not tool_calls:
                    answer = call_qwen(query)
                    console.print(Panel(answer, title="[bold green]Answer (direct)[/bold green]",
border_style="green"))
                    continue

                console.print(f"[cyan]Calling {len(tool_calls)} tool(s) via MCP...[/cyan]")
                tool_results = await execute_tools_via_mcp(session, tool_calls)
                answer = synthesize_answer(query, tool_results)
                console.print(Panel(answer, title="[bold green]Answer[/bold green]",
border_style="green"))

# -- SECTION 7: RUN -----

DEMO_QUERIES = [
    "What is the status of claim CL001?",
    "Is there any fraud risk on claim CL003?",
    "How much premium does customer C001 owe?",
    "Show me the details for policy P1002",
    "Please renew policy P1003",
]

if __name__ == "__main__":
    console.print("\n[bold magenta]=== MCP INSURANCE CAPSTONE ===[/bold magenta]")
    console.print("[dim]Real MCP protocol: stdio_client + ClientSession + call_tool[/dim]\n")

    test = call_qwen("Say OK in one word")
    if "ERROR" in test:
        console.print(Panel(test, title="[bold red]Ollama Connection Failed[/bold red]",
border_style="red"))
        exit(1)
```

```
console.print("[green]Ollama connected[/green]")
asyncio.run(run_insurance_agent(DEMO_QUERIES))
console.print("\n[bold yellow]=== Capstone Complete ===[/bold yellow]")
```

Sample Input / Output

```
Customer Query: "What is the status of claim CL001?"
Customer Query: "Is there any fraud risk on claim CL003?"

--- Output ---
MCP handshake complete (initialize)
Tools discovered (list_tools): ['get_customer', 'get_policy',
'get_claim_status', 'get_premium_due', 'run_fraud_check', 'renew_policy']

MCP call_tool(get_claim_status, {'claim_id': 'CL001'})
Result: Claim CL001: Status=Approved, Amount=Rs 25,000, Customer=C001, Policy=P1001

Answer: Your claim CL001 has been approved for Rs 25,000.
No further action is needed on your part at this time.
```

Key Concepts Covered

Concept	Description
Real MCP Protocol	stdio_client + ClientSession implement the actual initialize / list_tools / call_tool sequence, not direct Python calls.
Dynamic Tool Discovery	available_tools is built at runtime from list_tools() — adding a 7th server tool requires zero client changes.
LLM as Tool-Selector	Qwen reads the user query and available tool descriptions, then emits a JSON array of {tool, input} calls.
Transport Abstraction	Swapping stdio_client for an HTTP/SSE client moves the server to a different machine with almost no code change.

Cheat Sheet

Syntax / Call	Purpose
<code>FastMCP("name")</code>	Create a new MCP server instance.
<code>@mcp.tool()</code>	Register a function as a callable tool (an action).
<code>@mcp.resource("scheme://path")</code>	Register a function as read-only data, addressed by URI.
<code>@mcp.prompt()</code>	Register a function returning a reusable instruction template.
<code>mcp.run()</code>	Start the server and listen for client connections.
<code>StdioServerParameters(command, args)</code>	Describe how to launch the server subprocess for a stdio connection.
<code>stdio_client(server_params)</code>	Open a stdio transport to a (sub)process-based MCP server.
<code>ClientSession(read, write)</code>	Wrap a transport in a client session object.
<code>session.initialize()</code>	Perform the MCP handshake with the server.
<code>session.list_tools()</code>	Discover all tools the server currently exposes.
<code>session.call_tool(name, arguments=...)</code>	Invoke a specific tool by name with a dict of arguments.

Industry Use Cases

Pattern	Real-World Application
Tool registry / discovery	Plugin marketplaces, internal tool catalogs for enterprise AI assistants.
Tool schema design	API contract design for any system an LLM will call (function calling, agents).
MCP server with Tools	Exposing CRM/ERP/insurance/banking operations safely to an AI agent.
MCP Resources	Read-only knowledge bases, FAQs, and reference data surfaced to an LLM without write access.
MCP Prompts	Standardising how an organisation wants an LLM to handle recurring workflows (claims review, support tone).
Full client-server capstone	Production-style AI copilots integrated with real backend systems (insurance, healthcare, finance).

Troubleshooting

Issue	Solution
ModuleNotFoundError: mcp	Run <code>pip install mcp</code> (or <code>pip install -r requirements3.txt</code>).
Client hangs on <code>session.initialize()</code>	Confirm the server script path is correct and the server starts without errors on its own.
<code>call_tool()</code> raises an error for a valid tool name	Check that 'arguments' matches the tool's expected parameter names exactly.
LLM (Qwen) returns invalid JSON for tool selection	Tighten the prompt's 'Return ONLY JSON' instruction; strip markdown fences before <code>json.loads()</code> .
New server tool not appearing on the client	Restart the client — <code>list_tools()</code> is called fresh each run, so changes require a new session.
Cannot connect to Ollama (capstone)	Ensure 'ollama serve' is running and 'ollama pull qwen2.5:3b' has completed.

Conclusion

Across these eight labs, students progressed from a plain Python function with no protocol, through simulated tool registries and schema design, to building and running real MCP servers with Tools, Resources, and Prompts. The capstone tied everything together with a genuine MCP client-server connection driven by a local LLM — demonstrating how MCP lets an AI agent discover and safely use real business capabilities without any custom, one-off integration code.